AI Maze Solving

Nikhil Gupta

Introduction

The ability to navigate complex, obstructed environments is a foundational problem in the field of artificial intelligence. From robotics and logistics routing to character movement in simulated environments, the challenge of finding an optimal path from a starting state to a goal state remains highly relevant. Mazes serve as an ideal, abstract testing ground for these pathfinding problems, taking out complexities like real-world physics to focus on a finite, discrete grid. This project explores the efficacy, scalability, and computational trade-offs of various artificial intelligence methodologies in solving mazes. Specifically, this report compares traditional uninformed search algorithms, informed heuristic-based search, and a modern reinforcement learning approach to evaluate how different eras of AI solve the exact same spatial problem.

The theoretical background of pathfinding is rooted in graph theory and tree-search algorithms. Early approaches, such as Breadth-First Search (BFS) and Depth-First Search (DFS), operate as uninformed algorithms. They explore the state space systematically but lack inherent knowledge about the goal's location. While BFS guarantees shortest-path optimality in unweighted grids by expanding level by level, it suffers from massive memory consumption and

inefficiency as the state space grows. Conversely, DFS minimizes memory usage but sacrifices path optimality, often diving deep into dead ends. To overcome the computational bloat of uninformed search, informed algorithms like A* were developed. By introducing a heuristic function, such as the Manhattan distance, A* estimates the remaining cost to the goal, allowing the algorithm to intelligently prioritize which nodes to expand. This significantly reduces the search space while maintaining mathematical guarantees of optimality.

However, traditional search algorithms require a fully observable environment; the agent must have prior access to the entire grid layout to calculate its path. Modern reinforcement learning (RL) introduces a radically different approach: sequential decision-making in unmapped environments. Through tabular Q-Learning, an agent interacts with the maze through trial and error, receiving penalties for hitting walls and rewards for finding the exit. Using the Bellman equation, the agent updates a state-action value table (Q-table) over thousands of episodes, eventually learning an optimal policy without ever "seeing" the maze in its entirety. Comparing this learning-based approach to traditional search highlights a shift in AI, moving from deterministic planning to dynamic, reward-driven adaptation.

The methodology and testing framework for this project were heavily inspired by academic literature surrounding procedural environment generation and algorithmic benchmarking. In their research on maze solving and generation, Osmanković et al. highlighted the necessity of using randomized algorithmic generation to prevent bias when evaluating pathfinding agents.

Hand-drawn mazes inherently contain human patterns that algorithms might inadvertently exploit. To ensure an unbiased evaluation, this project adopts the principles outlined by Osmanković, utilizing a randomized implementation of Prim's Algorithm to procedurally generate perfect, fully connected maze structures. This guarantees that the evaluated algorithms are tested against complex, computationally challenging environments rather than trivial, human-designed puzzles.

By developing a custom data pipeline that translates visual .bmp images into navigable graph structures, this project provides a direct, head-to-head comparison of these AI paradigms. The subsequent sections of this report will detail the specific implementations of these algorithms, the custom Gymnasium environment designed for the Q-Learning agent, and a systematic evaluation of their execution time, node expansion, and scalability across increasingly massive grid environments.

Methods

To evaluate the performance of different artificial intelligence pathfinding algorithms, this project utilized a custom workflow to generate, parse, and solve maze environments. The methodology is divided into three primary components: the procedural generation and data

processing pipeline, the implementation of traditional graph-search algorithms, and the architecture of a custom reinforcement learning environment. External software libraries, including NumPy for matrix operations, OpenCV for image processing, and Gymnasium for the reinforcement learning API, were heavily utilized to construct the testing framework.

Environment Generation and Data Pipeline

A fundamental requirement for unbiased algorithmic evaluation is the use of procedurally generated, rather than hand-designed, test environments. To accomplish this, a custom procedural generation script was implemented using a randomized variant of Prim's Algorithm. Starting with a grid of impenetrable walls, the algorithm randomly carves out paths, ensuring that the resulting maze is perfectly connected—meaning there are no isolated sections and exactly one optimal path between any two points. These mazes are generated at varying odd-numbered dimensional scales and saved as bitmap (.bmp) image files, where black pixels represent walls and white pixels represent traversable paths.

To bridge the gap between visual data and mathematical computation, an environment parsing module was constructed using OpenCV. The MazeEnv class loads the .bmp images in grayscale and applies binary thresholding to convert the pixel data into a two-dimensional NumPy array. In this matrix representation, the visual maze is transformed into a navigable graph where walls are mapped to an integer value of 0 and paths to a value of 1. The environment scans the top

and bottom rows of the matrix to identify the entrance and exit nodes, effectively charting the visual image into a formal state space that any discrete search algorithm can traverse.

Traditional Search Implementations

With the state space formally defined, three traditional search algorithms were implemented as baselines for comparison: Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search. Both BFS and DFS operate as uninformed strategies. BFS utilizes a First-In-First-Out (FIFO) queue to explore the grid systematically by level. Because the grid is unweighted, meaning every movement costs exactly one step, BFS is mathematically guaranteed to return the absolute shortest path. Conversely, DFS employs a Last-In-First-Out (LIFO) stack, exploring deeply down individual corridors until reaching a dead end before backtracking. While DFS is highly memory-efficient, it does not guarantee path optimality.

To address the exhaustive nature of uninformed search, A* was implemented as an informed algorithm using a priority queue. A* evaluates nodes based on the function $f(n)=g(n)+h(n)$, where $g(n)$ is the cost from the start node to node n, and $h(n)$ is a heuristic estimating the cost from n to the goal. The algorithm was programmed to allow a choice between two heuristics: the Manhattan distance, calculated as $|x1-x2|+|y1-y2|$, and the Euclidean distance, calculated as $(x1-x2)2+(y1-y2)2$. Because the agent is restricted to orthogonal movement (up, down, left, right), the Manhattan distance provides a more accurate and admissible heuristic for this specific environment.

Reinforcement Learning Architecture

To compare traditional deterministic search against modern learning-based approaches, a Tabular Q-Learning agent was developed. Unlike BFS or A*, which analyze the complete graph structure, the reinforcement learning agent possesses no prior knowledge of the maze layout. To facilitate this interaction, the static MazeEnv was wrapped into a custom Gymnasium environment. The state space was flattened into a one-dimensional discrete space representing the agent's current coordinates, and the action space was defined as four discrete orthogonal movements.

The agent learns an optimal policy through interaction and a carefully shaped reward function. To encourage efficient pathfinding, the agent receives a massive positive reward (+100) for reaching the exit. To penalize inefficiency and prevent the agent from wandering aimlessly, a step penalty (−1) is applied for every valid movement. Finally, attempting to move into a wall results in the agent remaining in its current state and receiving a moderate penalty (−5), discouraging it from repeatedly colliding with boundaries.

The Q-table, initialized with zeros, is iteratively updated over thousands of episodes using the Bellman equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma a' \max Q(s',a') - Q(s,a)]$$

where α represents the learning rate and γ represents the discount factor, heavily weighting long-term rewards. Action selection is governed by an ϵ-greedy strategy, where the agent begins by taking purely random exploratory actions (ϵ=1.0). As training progresses, ϵ decays, transitioning the agent from exploring the grid to exploiting the highest-valued actions in its updated Q-table.

Evaluation Methods

The performance of these algorithms was evaluated using a multi-phase benchmarking script, who's results can be seen in Figure 1. The primary metrics for comparison were execution time, the total number of nodes explored (for traditional search), the number of episodes required for convergence (for Q-Learning), and final path optimality. The first phase consisted of a head-to-head comparison across scaled grid sizes ranging from 11x11 to 71x71, observing how rapidly the state-space explosion degraded the performance of the Q-Learning agent. The second phase involved a large-scale stress test, dropping the RL agent and evaluating BFS, DFS, and A* on procedurally generated mazes scaling up to 5001x5001 to analyze computational efficiency at such sizes.

Maze Size	BFS Time	DFS Time	A* Time	BFS Nodes	DFS Nodes	A* Nodes	QL Time	QL Status
11x11	9.51E-05	0.00016642	7.61E-05	39	51	20	1.594744444	Optimal
31x31	0.00066185	0.00094557	0.00086689	325	286	189	19.02280378	Optimal
51x51	0.001999378	0.00140762	0.00108886	1150	835	419	60.64845037	Optimal
71x71	0.005474091	0.00472474	0.00204778	2434	2085	811	153.1216686	Optimal
101x101	0.012780905	0.0011549	0.01200438	4647	647	2701		Skipped (Too Large)
501x501	0.382868767	0.26818037	0.07440925	117975	93455	20531		Skipped (Too Large)
1001x1001	1.39703989	0.77051592	1.00268316	497405	269269	222143		Skipped (Too Large)
5001x5001	31.78433323	15.0084124	20.1509624	10925088	5268077	4286715		Skipped (Too Large)

Figure 1: Benchmark results of different algorithms scaled across different maze sizes

Results

The evaluation of the pathfinding methodologies was conducted in two distinct phases to capture both the learning efficiency of the reinforcement learning agent and the computational limits of the traditional search algorithms. The benchmark analyzed execution time, search space efficiency (measured in nodes explored), and path optimality across increasingly complex environments.

Phase 1: Reinforcement Learning vs. Traditional Search

Phase 1 pitted the Q-Learning agent against Breadth-First Search (BFS), Depth-First Search (DFS), and A* across moderately sized mazes ranging from 11x11 to 71x71. The empirical data reveals a stark computational contrast between deterministic graph traversal and trial-and-error reinforcement learning. While the traditional algorithms resolved these environments in fractions of a second (e.g., A* required only 0.002 seconds for the 71x71 grid), the Q-Learning agent required exponentially more time to compute.

However, despite the massive time discrepancy, the tabular Q-Learning architecture proved remarkably robust. Across all Phase 1 scales—11x11, 31x31, 51x51, and 71x71—the agent successfully converged on a policy that yielded a 100% optimal path when compared to the A* baseline. The training time scaled predictably with the state-space expansion: requiring 1.59 seconds for the 11x11 maze and growing to 153.12 seconds (approximately 2.5 minutes) for the

71x71 maze. This data highlights a fundamental tradeoff: while RL is highly adaptable and can learn optimal policies in unmapped environments, relying on random epsilon-greedy exploration to populate a Q-table is orders of magnitude more computationally expensive than utilizing algorithms equipped with full environmental observability.

Phase 2: Large-Scale Traditional Search Stress Test

Phase 2 suspended the Q-Learning agent to conduct a large-scale stress test on the traditional graph-search algorithms. Grid sizes were scaled from 101x101 up to an extreme 5001x5001 environment, the latter comprising over 25 million total grid cells.

Analyzing the number of nodes explored across these extremes reveals the power of informed heuristics, as seen in Figure 2. Across all tested sizes, BFS operated as a brute-force flood fill. In the massive 5001x5001 maze, BFS expanded over 10.9 million distinct nodes to mathematically guarantee the shortest route. A*, conversely, drastically pruned the search tree. Guided by the Manhattan distance heuristic, A* prioritized nodes that mathematically trended toward the exit coordinates, evaluating only 4.28 million nodes—a 60% reduction in search space compared to BFS.

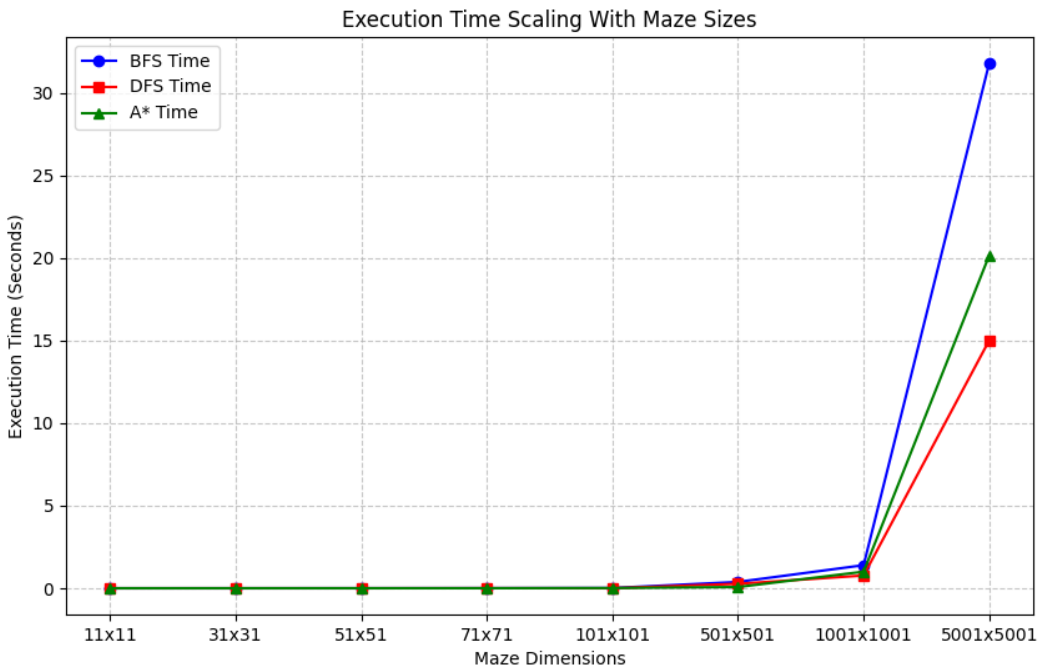

Figure 2: Execution time scaling of search algorithms across varying maze dimensions

Execution Time Scaling at Extremes

The execution time scaling corroborates the theoretical behaviors of these algorithms, while also revealing the hidden overhead of their underlying data structures. As expected, BFS required the longest execution time in the 5001x5001 grid, taking 31.78 seconds to manage the massive queue operations associated with its 10.9 million node expansions. A* required 20.15 seconds to solve the same maze.

Interestingly, DFS recorded the fastest raw execution time at this extreme scale, finishing in just 15.01 seconds while exploring 5.26 million nodes. This highlights a critical nuance in algorithmic implementation: the constant-time push and pop operations of a LIFO stack in DFS possess significantly lower computational overhead than maintaining the ordered priority queue (heap) required by A*. However, because DFS blindly plunges down paths, it consistently returns severely suboptimal path lengths. Therefore, while DFS may execute marginally faster in raw computation time under specific procedural generations, this data conclusively demonstrates that heuristic-driven A* search provides the optimal balance—drastically reducing the expanded search space while maintaining guaranteed path optimality at a large scale.

Conclusion

The objective of this project was to systematically evaluate and compare the efficacy, scalability, and computational trade-offs of traditional graph-search algorithms and reinforcement learning in navigating procedurally generated spatial environments. The results clearly demonstrate the operational boundaries and ideal use cases for each algorithm. By abstracting visual maze data into a discrete graph, this project provided a head-to-head analysis of Breadth-First Search, Depth-First Search, A* Search, and Q-Learning.

The primary strength of traditional uninformed algorithms, specifically BFS, is their guarantee of path optimality in unweighted grids. However, the evaluation demonstrated that this mathematical guarantee comes at an unsustainable computational cost in larger environments. DFS, while requiring less memory overhead and executing marginally faster at massive scales due to the constant-time operations of a Last-In-First-Out stack, is fundamentally flawed for true pathfinding, as its blind exploration consistently yields severely suboptimal routes. The data conclusively proved that informed search, specifically A* utilizing the Manhattan distance heuristic, offers the optimal deterministic solution. A* successfully balanced computational efficiency with path optimality, reducing the expanded search space by over 60% compared to BFS in the largest procedural environments. However, the critical weakness of all these traditional algorithms is their absolute reliance on environmental observability; they require a complete, prior mathematical map of the maze to function.

Reinforcement learning offers a powerful model shift, trading deterministic planning for interaction-driven adaptation. The primary strength of the Q-Learning agent was its ability to converge on a perfectly optimal policy without any prior knowledge of the maze layout, relying entirely on a shaped reward structure and sequential decision-making. Nevertheless, the benchmarking data exposed the severe scalability limitations of tabular Q-Learning. As the maze dimensions increased, the discrete state space expanded exponentially, rendering the random epsilon-greedy exploration strategy computationally impossible. While A* could resolve a 71x71 maze in milliseconds, the Q-Learning agent required nearly 3 minutes of simulated experience to populate its state-action value table for the exact same environment.

Future iterations of this project could address these specific limitations by transitioning from tabular reinforcement learning to deep reinforcement learning. Rather than maintaining a discrete, two-dimensional array for the Q-table, Deep Q-Network (DQN) could be implemented using PyTorch. By utilizing a Convolutional Neural Network (CNN) as a function approximator, the agent could process the maze grid visually, identifying generalized spatial features such as dead ends and corridor structures. This would allow the agent to generalize its navigational policy to new, unseen mazes without requiring thousands of episodes to relearn absolute grid coordinates. Ultimately, while heuristic-driven A* remains the definitive solution for static, fully observable pathfinding, advancing to deep reinforcement learning represents the most promising step forward for scaling AI in complex, unmapped environments.

Works Cited

[1] D. Osmanković and S. Konjicija, "Implementation of Q - Learning algorithm for solving maze problem," 2011 Proceedings of the 34th International Convention MIPRO, Opatija, Croatia,

2011, pp. 1619-1622.